Improving Human Detection in Soccer Video with Neighboring Frames

Testing whether neighboring frames improve YOLO11n detections

Jake Bohane

October 5, 2026

Table of contents

1 Introduction **1**
- 1.1 Architecture . . . 2
- 1.2 Experiment information . . . 4
  - 1.2.1 Architecture changes . . . 4
  - 1.2.2 Trial details . . . 4
  - 1.2.3 Metric used . . . 5

2 Do neighboring frames improve detection? **5**
- 2.1 Comparison with the soccer adapted YOLO baseline . . . 5
- 2.2 Comparison with repeated frame controls . . . 7

3 Does more complex multi frame processing help? **9**

4 Conclusion **10**

References **10**

1 Introduction

When I first applied a pretrained YOLO detector to soccer footage, I assumed that processing video meant the model used information from neighboring frames. Instead, the detector processed each frame independently. A separate tracker connected detections over time, but information from neighboring frames did not feed back into the detector's predictions.

This raised a question: could information from surrounding frames improve detection in the current frame? Soccer footage provides a natural setting for this experiment because players move continuously, while individual frames can contain motion blur, overlapping, and small objects. Neighboring frames might provide useful context when a single frame is ambiguous.

1.1 Architecture

To investigate this question, I used YOLO11n, a lightweight, general purpose object detector (Ultralytics 2024). It was pretrained on COCO, a dataset of everyday scenes containing people, vehicles, animals, and other common objects (Lin et al. 2014), rather than specifically soccer footage. It therefore recognizes players, goalkeepers, and referees under the same general "person" category.

YOLO11n processes an image through three main components: the backbone, the neck, and the detection head. The backbone applies a sequence of layers that transforms the image into tensors, progressively reducing their spatial dimensions while generally increasing their number of channels. The neck applies layers that upsample and downsample tensors, combine tensors from different backbone stages, and adjust their channel counts. It produces three output tensors at different spatial resolutions. The detection head uses each tensor to predict bounding box coordinates and class probabilities at its respective resolution. For this study, postprocessing retains predictions for the person class, filters low confidence predictions, and removes duplicate boxes.

The following diagrams show the model path from backbone to neck to detection head. Each labeled layer represents a major category of transformation in the model, such as convolution, C3k2 feature processing, spatial pooling, or C2PSA attention. Supporting operations inside that module, such as BatchNorm, SiLU activation, pooling, or attention sublayers, are included under the same label. Tensor faces are proportional to spatial dimensions, while depth represents channel count. The neck and detection head use larger spatial drawing scales so their connections remain readable.

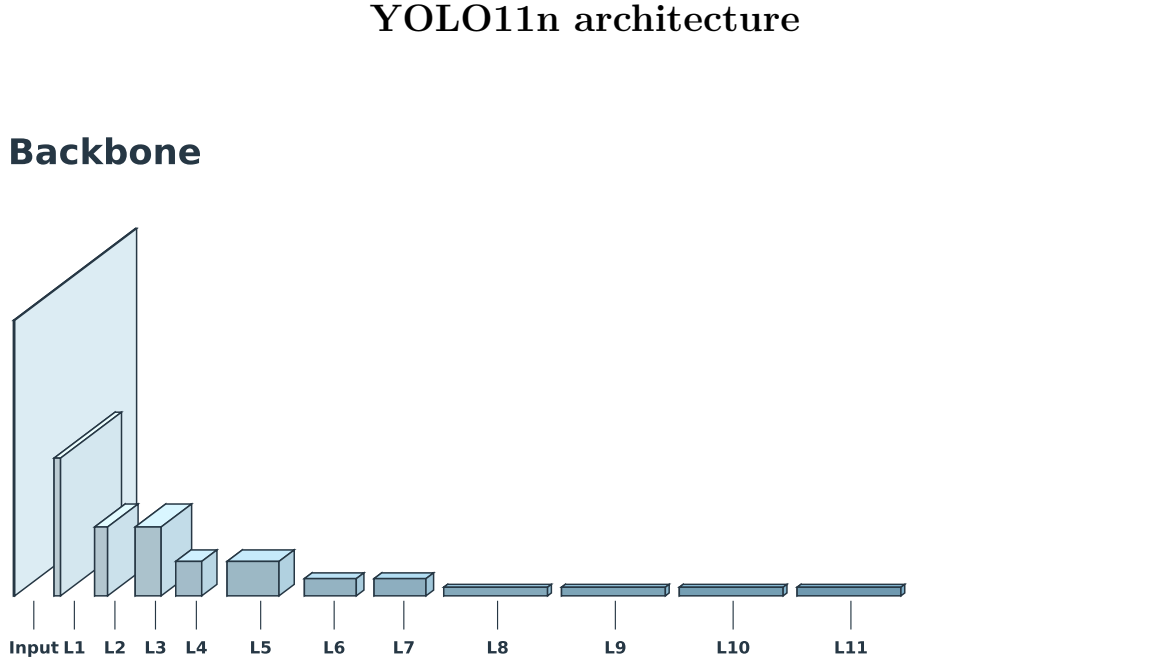


Figure 1: Backbone modules shown as sequencial tensors.

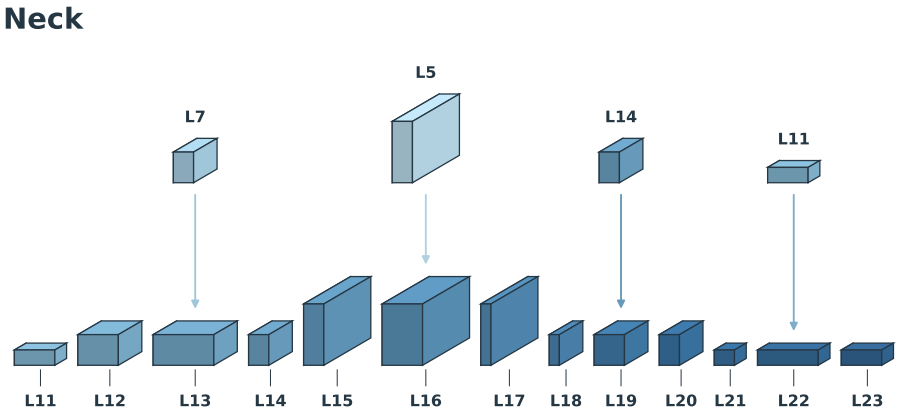


Figure 2: Neck modules shown as sequential tensors beginning with backbone output L11. Reused tensors appear above the layer that receives them through concatenation, connected by downward arrows.

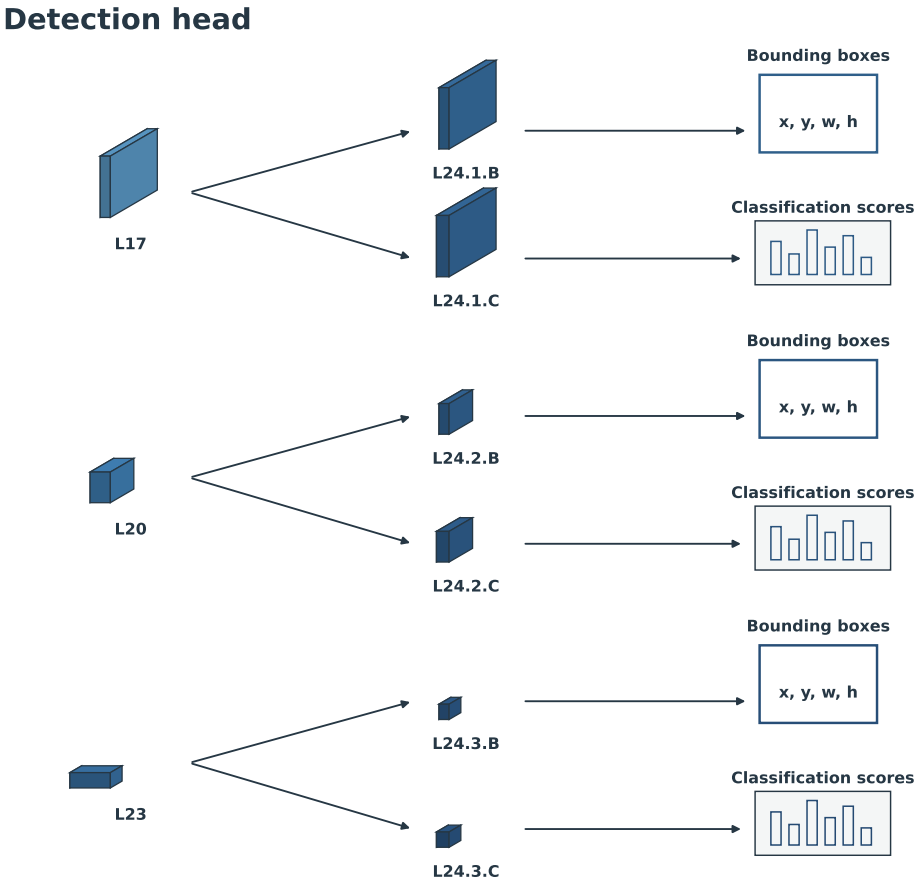


Figure 3: Detection head modules at three resolutions, each with separate branches for box coordinates and class probabilities.

1.2 Experiment information

1.2.1 Architecture changes

For the multi frame models, the previous, current, and next frames each pass through the same backbone and neck. At each of the three output resolutions, using layers L17, L20, and L23, their tensors are concatenated along the channel dimension. We then apply the selected multi frame processing layer(s) before passing the resulting tensors to the detection head, which predicts detections for the current frame. The multi frame variations test different layer designs after concatenation.

The chart labels describe the processing applied after concatenation. The three frame 1×1 model uses a multi frame processing layer containing a 1×1 convolution, BatchNorm, and SiLU activation to reduce the concatenated tensor from three times the original channel count back to the original count. The three frame 1×1 plus 3×3 model uses the previously stated 1x1 layer followed by a 3×3 convolution, BatchNorm, and SiLU activation layer. The five frame model uses five neighboring frames and applies the same 1×1 plus 3×3 sequence. Additional variants replace the 3×3 convolution within this layer with a 5×5 convolution or use a C3k2 layer. The repeated frame controls use the same multi frame processing layers but provide three copies of the current frame instead of neighboring frames.

To test whether neighboring frames provide useful information, we compare the main multi frame architectures with matching controls that concatenate the current frame's tensors three times. We also compare them with the original YOLO11n architecture. All models are trained on the same soccer data.

All models start from the pretrained YOLO11n checkpoint. The backbone, neck, and detection head therefore begin with pretrained weights. The backbone and neck remain frozen throughout training, and their BatchNorm statistics remain fixed, so the feature extraction process does not change. The original YOLO11n baseline trains only its pretrained detection head. The multi frame models fine tune their pretrained detection heads and train the newly added multi frame processing layers, which are randomly initialized.

1.2.2 Trial details

The training data contains 750 labeled frames from six short clips across three games. The development validation data contains 1,125 labeled frames from nine clips across three different games. The footage and annotations come from the SoccerNet SN GSR 2024 dataset (SoccerNet 2024). Every human annotation, including players, goalkeepers, and referees, is evaluated as the single person class. Frames are resized to 640 by 384 pixels before entering the model.

Each architecture is trained twice, with random seeds 42 and 43. The seed controls sources of randomness such as the new trainable weights and the order of training examples. Reporting both runs shows whether a result holds across two starting conditions, although two seeds are not enough for a formal statistical conclusion.

1.2.3 Metric used

I evaluate detection quality with AP50 to 95, following the average precision framework used in object detection benchmarks (Lin et al. 2014). This metric summarizes how well the model finds people and draws accurate boxes around them. Higher values indicate better detection performance.

More specifically, the evaluator compares each predicted box with the labeled boxes and measures their overlap percentage. The overlap percentage is the area shared by the two boxes divided by the total area covered by either box, expressed as a percentage. At each overlap threshold from 50 percent through 95 percent, a prediction counts as correct when its overlap with a labeled box meets or exceeds that threshold. Otherwise, it counts as incorrect.

For each overlap threshold, the evaluator uses the same predictions ranked from highest to lowest confidence. It then moves through this list, adding one prediction at a time and plotting the resulting precision and recall on the precision recall curve. AP averages the model's precision at 101 recall levels from 0.00 to 1.00. Because the curve does not usually contain a point at every exact recall level, the evaluator uses the highest precision observed at that recall level or any higher recall level. This smooths out temporary fluctuations in the curve before the precision values are averaged.

AP50 to 95 repeats this process for all ten overlap thresholds and averages the resulting AP values. The metric therefore evaluates both whether people are found and how accurately their boxes are placed. It is not simply the percentage of people detected.

2 Do neighboring frames improve detection?

The following comparisons use all 1,125 validation targets. All models completed 10 training epochs. Each pair of bars shows seeds 42 and 43 separately. The lighter bar on the left represents seed 42, and the darker bar on the right represents seed 43.

2.1 Comparison with the soccer adapted YOLO baseline

This first comparison asks whether the separately trained multi frame models perform better than a soccer adapted YOLO11n detector without the added multi frame processing layer. It establishes the overall performance difference, but it cannot by itself separate the value of neighboring frames from the value of the added trainable layer.

The baseline uses the original YOLO11n architecture with its detection head fine tuned on the soccer data. The backbone and neck, including their BatchNorm statistics, remain frozen. The multi frame models also train the added layers that combine information from neighboring frames.

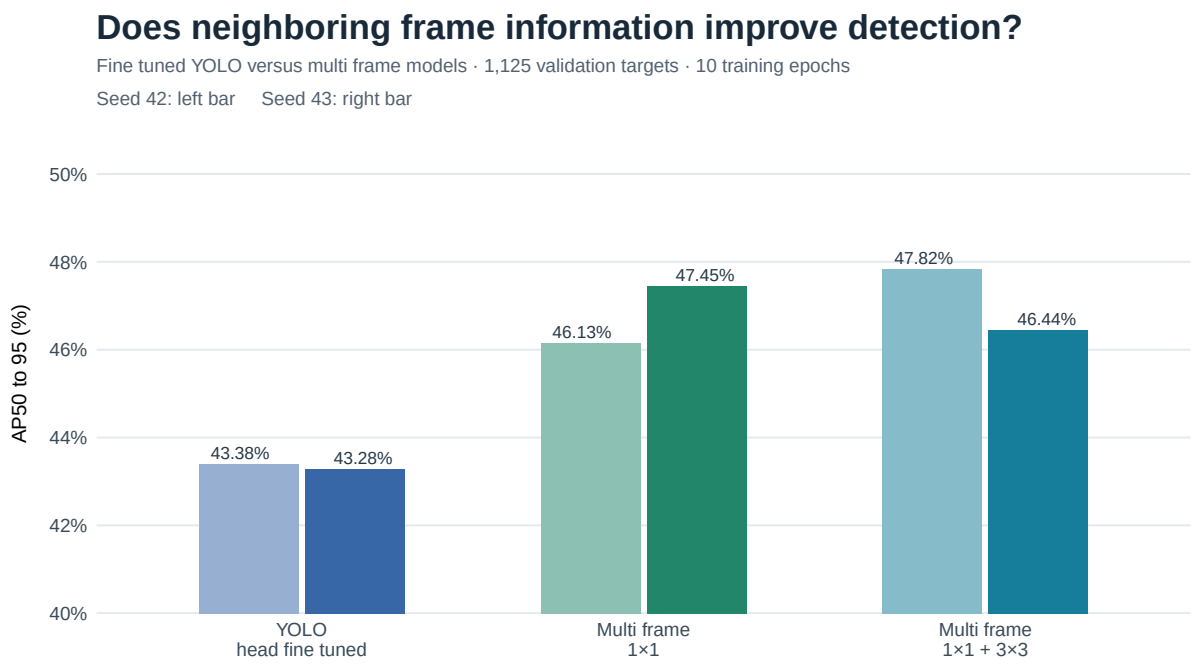


Figure 4: Detection performance of the soccer adapted YOLO baseline and the two main multi frame architectures.

Model	Seed.42	Seed.43	Average
YOLO head fine tuned	43.38%	43.28%	43.33%
Multi frame 1×1	46.13%	47.45%	46.79%
Multi frame 1×1 + 3×3	47.82%	46.44%	47.13%

Both multi frame models score higher than the soccer adapted YOLO11n baseline. The multi frame 1×1 model has an average AP50 to 95 of 46.79 percent, compared with 43.33 percent for the baseline, a difference of 3.46 percentage points. The 1×1 followed by 3×3 model has a score of 47.13 percent, compared with 43.33 percent for the baseline, a difference of 3.80 points. These comparisons show that the separately trained multi frame models performed better than the head only fine tuned baseline. Because the multi frame models also add a trainable layer, the baseline comparison alone cannot determine whether the improvement came from the neighboring frames or from the added processing. The matched repeated frame controls provide that comparison in the next section.

2.2 Comparison with repeated frame controls

This second comparison asks whether the performance difference is due specifically to neighboring frames. It compares each multi frame model with a separately trained model that has the same added layer but receives three copies of the current frame.

Each control has the same architecture as its corresponding multi frame model, but receives the current frame three times during both training and evaluation. Within each seed, the paired models share their initial weights, training data, and sample order. This comparison tests the contribution of neighboring frame information while holding the architecture fixed.

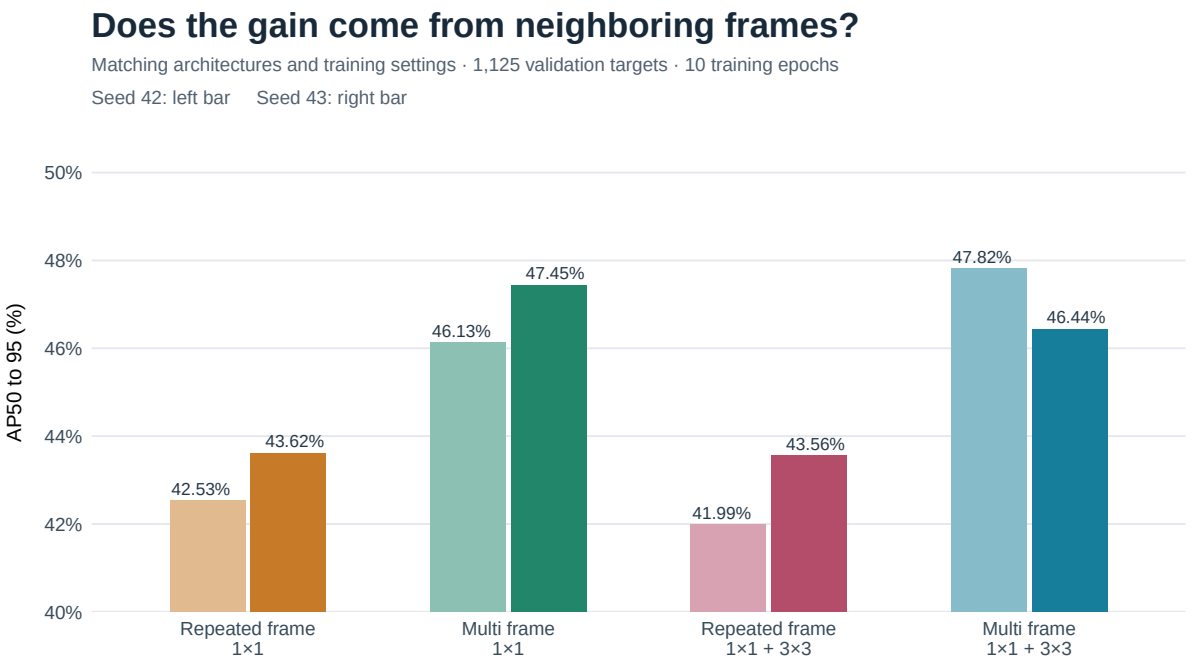


Figure 5: Multi frame models compared with matching repeated frame controls.

Model	Seed.42	Seed.43	Average
Repeated frame 1×1	42.53%	43.62%	43.08%
Multi frame 1×1	46.13%	47.45%	46.79%
Repeated frame 1×1 + 3×3	41.99%	43.56%	42.77%
Multi frame 1×1 + 3×3	47.82%	46.44%	47.13%

The multi frame models also outperform their matched repeated frame controls. The 1×1 multi frame model has an average AP50 to 95 of 46.79 percent, compared with 43.08 percent for its repeated frame control, a difference of 3.71 percentage points. The 1×1 followed by 3×3 multi frame model has a score of 47.13 percent, compared with 42.77 percent for its repeated frame control, a difference of 4.36 points. These comparisons show that the separately trained multi frame models performed better than models with the same added processing that received three copies of the current frame. Because the paired models use the same trainable architecture and differ

in whether they receive neighboring frames or repeated current frames, the comparison provides evidence that the neighboring frame information contributed to the higher scores.

3 Does more complex multi frame processing help?

The multi frame models add one multi frame processing layer after the neck and before the detection head. The following comparison changes the design of that layer and, for the five frame variants, the number of input frames.

This comparison uses the same 1,116 validation targets for every architecture. Nine targets are excluded because they are too close to the start of a clip to provide two preceding frames. Scores therefore differ slightly from the previous section. The five frame variants also use 744 training targets rather than 750 because of the same boundary requirement.

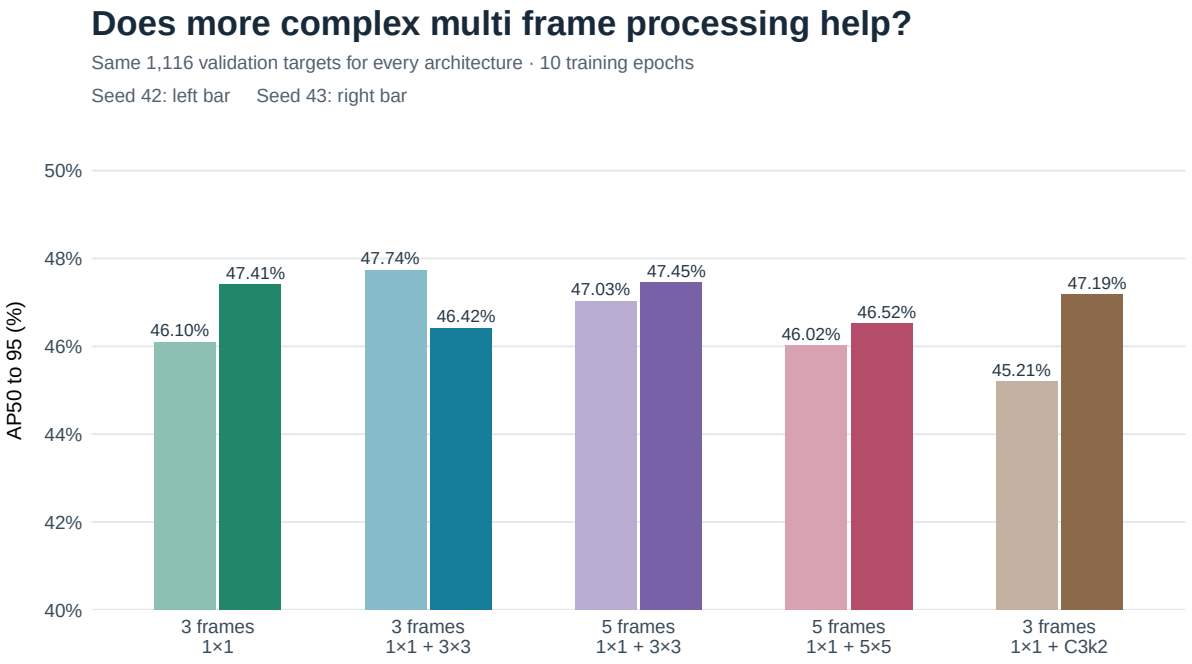


Figure 6: Comparison of five multi frame architectures across two training seeds.

Model	Seed.42	Seed.43	Average
3 frames 1×1	46.10%	47.41%	46.75%
3 frames 1×1 + 3×3	47.74%	46.42%	47.08%
5 frames 1×1 + 3×3	47.03%	47.45%	47.24%
5 frames 1×1 + 5×5	46.02%	46.52%	46.27%
3 frames 1×1 + C3k2	45.21%	47.19%	46.20%

The five frame model with 1×1 and 3×3 convolutions has the highest average AP50 to 95 score at 47.24 percent. Its lead over the three frame 1×1 followed by 3×3 model is only 0.16 percentage points, and the remaining architectures are also close. Performance also changes meaningfully between the two seeds. With only two trials, that variability makes it difficult to tell whether the small average differences reflect an architectural advantage or ordinary variation in training. More

seed trials and a broader training set from additional games would be needed to identify a better temporal processing architecture.

4 Conclusion

This report asked two main questions. First, can information from neighboring video frames improve detection of annotated people in soccer footage? Second, if it can, does a more complicated multi frame architecture provide a further advantage?

The results answer the first question positively. In these experiments, the multi frame models achieved higher average AP50 to 95 scores than both the head only fine tuned YOLO11n baseline and the matching controls that repeated the current frame. Together, the comparisons in Sections 2.1 and 2.2 indicate that the improvement came from providing neighboring frames to the model, rather than from simply adding trainable layers.

The experiments did not identify an ideal multi frame processing architecture. The five frame model with 1×1 and 3×3 convolutions had the highest average score, but the differences between architectures were small and varied across seeds. More training trials and a larger, more varied set of training and validation data would be needed to select a reliable multi frame processing architecture.

These findings do not indicate an error in the original YOLO11n design. YOLO11n is a general purpose detector with pretrained weights that can be fine tuned on task specific data. This project used that transfer learning setup to test whether neighboring frame information could make the detector more useful for soccer footage. Freezing the backbone and neck while training the detection head and added multi frame processing layers was a choice made for this experiment, rather than a claim about how the original model must be used. The result supports neighboring frame input as a promising direction while leaving the best implementation unresolved.

References

Lin, Tsung-Yi, Michael Maire, Serge Belongie, et al. 2014. "Microsoft COCO: Common Objects in Context." *European Conference on Computer Vision*, 740–55. https://doi.org/10.1007/978-3-319-10602-1_48.

SoccerNet. 2024. *SN GSR 2024: SoccerNet Game State Reconstruction Dataset.* https://huggingface.co/datasets/SoccerNet/SN-GSR-2024.

Ultralytics. 2024. *Ultralytics YOLO11.* https://docs.ultralytics.com/models/yolo11/.